Proyecto Final

DESARROLLO DE PROYECTOS DE APRENDIZAJE AUTOMATICO ML PARA CIENCIA DE DATOS

Alumno

Emmanuel Caleb Pérez Estrada

Universidad Autónoma de Aguascalientes

al243819@edu.uaa.mx

Profesor

Luis Eduardo Bautista Villalpando

Aguascalientes, AGS; Viernes 8 de mayo del 2026

Sistema Inteligente de Analítica de Aprendizaje Basado en Machine Learning, Elasticsearch y Kibana para el Análisis del Comportamiento Estudiantil

line 1: Emmanuel C. Pérez Estrada
line 2: *Maestría en Informática y Tecnologias Computacionales*
line 3: *Universidad Autónoma de Aguascalientes*
line 4: Aguascalientes, México
line 5: al243819@edu.uaa.mx

***Abstract*— This paper presents the development of an intelligent learning analytics system focused on the analysis and visualization of student interaction behavior through Machine Learning and Elastic Stack technologies. The project was based on the creation and enrichment of a dataset composed of interaction variables such as session time, clicks, keystrokes, inactivity periods, and activity levels, allowing the identification of behavioral patterns associated with student engagement. Data preprocessing and feature engineering techniques were applied to generate additional variables, including engagement score, interaction rate, and inactivity ratio, improving the analytical capacity of the dataset.**

For the Machine Learning stage, feature selection was performed using the Relief Algorithm to identify the most relevant variables for the classification process. Subsequently, a Decision Tree model was implemented to classify student activity levels into low, medium, and high categories, enabling the interpretation of behavioral tendencies from interaction data. The results obtained from the model were later integrated into Elasticsearch, allowing efficient storage and indexing of the generated information.

Additionally, Kibana was used to develop interactive dashboards capable of visualizing behavioral metrics, activity distributions, engagement indicators, and interaction trends in real time. Docker containers were utilized to deploy Elasticsearch and Kibana services within a virtualized Linux environment, facilitating portability and reproducibility within the MLOps workflow.

Keywords*— *learning analytics, machine learning, decision tree, Elasticsearch, Kibana, Docker, MLOps, student behavior analysis, data visualization, feature engineering, educational analytics

INTRODUCCIÓN

En la actualidad, el crecimiento acelerado de las tecnologías digitales dentro de los entornos educativos ha generado una gran cantidad de información relacionada con la interacción de los estudiantes en plataformas computacionales, permitiendo registrar patrones de comportamiento que anteriormente resultaban difíciles de analizar de manera sistemática., dichas interacciones incluyen acciones como pulsaciones de teclado, movimientos y clics del mouse, tiempos de sesión, periodos de inactividad y diferentes métricas asociadas al comportamiento del usuario durante la realización de actividades académicas. A partir de este contexto, surge la necesidad de desarrollar soluciones capaces de transformar estos datos en información útil para comprender el nivel de participación, interacción y compromiso de los estudiantes dentro de entornos digitales de aprendizaje.

Dentro del área de Learning Analytics, el uso de técnicas de Machine Learning ha cobrado una gran relevancia debido a su capacidad para identificar patrones complejos y generar modelos predictivos que facilitan la interpretación del comportamiento estudiantil por lo que permiten analizar grandes volúmenes de datos provenientes de las interacciones de los usuarios y construir modelos capaces de clasificar o predecir diferentes indicadores relacionados con el aprendizaje, el rendimiento académico o el nivel de actividad de los estudiantes, con base de eso la incorporación de procesos de preparación y enriquecimiento de datos resulta fundamental para mejorar la calidad del análisis, ya que permite generar nuevas variables derivadas que representan de manera más precisa el comportamiento de los usuarios dentro del sistema.

Por otro lado, dentro del enfoque MLOps, no solo resulta importante el desarrollo del modelo de Machine Learning, sino también la integración de herramientas que permitan gestionar el ciclo de vida completo del sistema, incluyendo el almacenamiento de datos, la visualización de resultados y la implementación de entornos reproducibles, por lo que tecnologías como Elasticsearch y Kibana permiten construir plataformas de análisis y monitoreo capaces de almacenar grandes volúmenes de información y representarlos mediante dashboards interactivos, facilitando la exploración visual de patrones y tendencias presentes en los datos, a través del uso de Docker contribuye a la creación de entornos portables y consistentes, permitiendo desplegar servicios de forma más eficiente dentro de arquitecturas modernas de análisis de datos.

El presente proyecto tiene como objetivo desarrollar un sistema inteligente de analítica de aprendizaje orientado al análisis del comportamiento estudiantil mediante técnicas de Machine Learning y herramientas del Elastic Stack, por eso se

construyó un conjunto de datos basado en variables de interacción como tiempo de sesión, clics, pulsaciones de teclado y periodos de inactividad, sobre el cual se aplicaron procesos de feature engineering para enriquecer la capacidad analítica del sistema, después se implementó un modelo de Árbol de Decisión apoyado por el algoritmo Relief para la selección de características relevantes, permitiendo clasificar el nivel de actividad de los estudiantes en diferentes categorías, ya al final los resultados obtenidos fueron integrados en Elasticsearch y visualizados mediante dashboards desarrollados en Kibana, permitiendo representar gráficamente diferentes métricas relacionadas con el comportamiento estudiantil y facilitando el análisis interactivo de la información lo que el proyecto demuestra cómo la combinación de Machine Learning, Elastic Stack y técnicas de visualización de datos puede contribuir al desarrollo de soluciones orientadas a la analítica educativa y al monitoreo inteligente de la interacción de los estudiantes dentro de entornos digitales..

DESARROLLO

Preparación y enriquecimiento de los datos

El desarrollo del proyecto comenzó con la construcción de un conjunto de datos orientado al análisis del comportamiento de estudiantes dentro de entornos digitales de interacción. Para ello, se definieron variables relacionadas con la actividad del usuario durante una sesión de trabajo, considerando métricas como el tiempo de sesión, la cantidad de clics realizados, las pulsaciones de teclado y los periodos de inactividad. Estas variables fueron seleccionadas debido a su capacidad para representar distintos patrones de interacción y participación dentro del sistema, permitiendo modelar diferentes niveles de actividad del estudiante a partir de sus acciones registradas durante el uso de la plataforma.

Durante las primeras etapas del proyecto, se identificó que una relación excesivamente directa entre algunas variables y la clase objetivo generaba modelos demasiado simples, en los cuales el algoritmo dependía casi por completo de una sola característica para tomar decisiones con el objetivo de evitar este comportamiento y generar un sistema más representativo, se llevó a cabo un proceso de enriquecimiento del dataset mediante técnicas de feature engineering, incorporando nuevas variables derivadas capaces de describir de manera más precisa el comportamiento de los usuarios dentro del entorno digital de las cuales las variables generadas se incluyeron métricas como el engagement score, diseñado para representar el nivel de interacción general del estudiante considerando clics, pulsaciones de teclado y periodos de inactividad; el interaction rate, utilizado para medir la intensidad de interacción durante la sesión; y el inactivity ratio, enfocado en representar la proporción de tiempo inactivo respecto al tiempo total de sesión, de igual forma, se creó una representación numérica del nivel de actividad con el fin de facilitar posteriores análisis y visualizaciones dentro de Kibana.

Este proceso permitió construir un conjunto de datos más equilibrado y con relaciones menos triviales entre las variables y la clase objetivo, favoreciendo la generación de modelos más complejos y representativos, por fin el dataset enriquecido fue exportado en formato CSV para su reutilización en las etapas posteriores del flujo de trabajo, incluyendo el entrenamiento del modelo, la integración con Elasticsearch y la visualización mediante dashboards interactivos.

Seleccion de caracteristicas y entrenamiento del modelo de clasificación

Una vez completada la preparación y enriquecimiento del conjunto de datos, se procedió a la etapa de selección de características y entrenamiento del modelo de Machine Learning, con el objetivo de identificar las variables más relevantes para la clasificación del nivel de actividad de los estudiantes; dentro de los procesos de análisis de datos, la selección de características representa una etapa fundamental, ya que permite reducir redundancias, mejorar la interpretabilidad del modelo y optimizar su capacidad de aprendizaje a partir de los datos disponibles.

Para llevar a cabo este proceso, se implementó el algoritmo Relief, técnica ampliamente utilizada dentro del área de selección de características debido a su capacidad para evaluar la relevancia de cada variable a partir de la diferencia entre instancias pertenecientes a distintas clases. A diferencia de otros métodos basados únicamente en estadísticas globales, Relief analiza el comportamiento local de los datos, permitiendo identificar qué atributos contribuyen de manera más significativa a distinguir entre diferentes niveles de actividad del estudiante, el uso del algoritmo permitió confirmar que las variables generadas durante el proceso de enriquecimiento del dataset aportaban información relevante para el modelo, evitando que la clasificación dependiera únicamente de una sola característica dominante, después se implementó un modelo de Árbol de Decisión para clasificar el nivel de actividad de los estudiantes en categorías de actividad baja, media y alta, se seleccionó debido a su capacidad para generar modelos interpretables y estructurados mediante reglas de decisión jerárquicas, facilitando la comprensión de cómo las diferentes variables influyen en el proceso de clasificación, dichas variables utilizadas durante el entrenamiento se incluyeron el tiempo de sesión, la cantidad de clics, las pulsaciones de teclado, el tiempo de inactividad, el engagement score, el interaction rate y el inactivity ratio.

Durante el entrenamiento del modelo, el conjunto de datos fue dividido en subconjuntos de entrenamiento y prueba, permitiendo evaluar el desempeño del clasificador sobre datos no utilizados previamente durante el aprendizaje, lo que permitió medir la capacidad de generalización del modelo y verificar su comportamiento frente a diferentes patrones de interacción estudiantil como resultado el Árbol de Decisión generó una estructura más equilibrada y compleja en comparación con las primeras versiones del proyecto, utilizando múltiples variables en distintos niveles del árbol para tomar decisiones de clasificación, asimismo el modelo permitió identificar relaciones importantes entre las variables de interacción y el nivel de actividad de los estudiantes, observándose que métricas como el engagement score y el inactivity ratio presentan una influencia significativa en la clasificación final, lo que este comportamiento demuestra que el enriquecimiento del dataset y la incorporación de nuevas variables derivadas contribuyeron directamente a mejorar la capacidad analítica del sistema y a generar un modelo más representativo del comportamiento estudiantil dentro de entornos digitales. Por último, la estructura interpretable del Árbol de Decisión facilitó el análisis de las reglas de clasificación generadas por el modelo, permitiendo comprender de manera más clara cómo diferentes combinaciones de interacción pueden relacionarse con

distintos niveles de actividad, lo que resulta especialmente importante dentro del contexto de Learning Analytics, donde no solo interesa obtener predicciones, sino también comprender los patrones de comportamiento presentes en los datos para apoyar futuros procesos de análisis y monitoreo estudiantil.

Integración con Git y GitHub para el control del proyecto

Dentro del enfoque MLOps, el control de versiones es una práctica esencial para garantizar la organización, trazabilidad y mantenimiento de los proyectos de Machine Learning, especialmente en escenarios donde el desarrollo del sistema involucra múltiples modificaciones sobre datasets, modelos y procesos de análisis, donde se utilizaron las herramientas Git y GitHub con el objetivo de gestionar de manera estructurada el código fuente, los archivos del proyecto y las distintas versiones generadas durante el desarrollo del sistema de analítica de aprendizaje.

El proceso comenzó con la creación y utilización de un repositorio remoto en GitHub, el cual permitió almacenar el proyecto dentro de un entorno centralizado y accesible para futuras actualizaciones, luego, el repositorio fue clonado dentro de un entorno Linux ejecutado mediante máquina virtual permitiendo trabajar de manera local sobre el proyecto y establecer una conexión directa con el repositorio remoto, lo que facilitó el manejo organizado de los archivos relacionados con el modelo de Machine Learning, el dataset enriquecido y los diferentes componentes utilizados durante el desarrollo de la práctica.

Durante el flujo de trabajo, se utilizaron comandos fundamentales de Git para monitorear el estado del repositorio, agregar archivos modificados al área de preparación y registrar cambios mediante commits descriptivos, con este proceso permitió mantener un historial claro de las modificaciones realizadas en el proyecto, facilitando la identificación de cambios importantes y fortaleciendo la trazabilidad del desarrollo. Asimismo, el uso de mensajes descriptivos en cada commit permitió documentar de manera más precisa las actualizaciones relacionadas con el dataset, el modelo y las configuraciones del entorno. Una vez registrados los cambios de manera local, se llevó a cabo la sincronización del proyecto con GitHub mediante el proceso de *push*, permitiendo actualizar el contenido almacenado en el repositorio remoto, durante esta etapa fue necesario utilizar un token de acceso personal para la autenticación del usuario, debido a las políticas actuales de seguridad implementadas por GitHub, las cuales reemplazan el uso de contraseñas tradicionales en operaciones de sincronización remota.

La integración entre Git y GitHub permitió fortalecer el flujo de trabajo dentro del enfoque MLOps, asegurando que el proyecto pudiera mantenerse organizado, respaldado y preparado para futuras modificaciones o despliegues, también el uso de control de versiones facilitó la gestión del código fuente y de los datasets utilizados durante el proyecto, contribuyendo a mejorar la reproducibilidad y trazabilidad de los experimentos realizados durante el desarrollo del modelo de Machine Learning.

Implementación de Docker y Elastic Stack

Con el propósito de construir un entorno más cercano a arquitecturas reales de analítica y monitoreo de datos, se llevó a cabo la implementación de Docker como herramienta de contenedorización para desplegar los servicios de Elasticsearch y Kibana dentro de un entorno Linux ejecutado mediante máquina virtual, con el uso de Docker permitió encapsular los diferentes componentes del sistema dentro de contenedores independientes, facilitando su instalación, configuración y ejecución sin depender directamente de las características específicas del sistema operativo anfitrión. Para la implementación de Elastic Stack, se utilizó un archivo docker-compose.yml, el cual permitió definir y gestionar los contenedores de manera centralizada Elasticsearch fue empleado como motor de almacenamiento, búsqueda y análisis de datos, mientras que Kibana se utilizó como plataforma de visualización interactiva para representar gráficamente la información generada por el modelo de Machine Learning, gracias al uso de contenedores facilitó considerablemente el despliegue de ambos servicios, permitiendo construir una arquitectura funcional de análisis de datos con una configuración relativamente sencilla.

Durante el proceso de implementación fue necesario realizar ajustes relacionados con la asignación de recursos dentro de la máquina virtual, particularmente en la memoria RAM disponible para Elasticsearch, debido a los requerimientos del servicio para su correcto funcionamiento, ya realizadas las configuraciones necesarias, ambos contenedores lograron ejecutarse correctamente, permitiendo acceder a Elasticsearch mediante el puerto 9200 y a Kibana mediante el puerto 5601, que posteriormente, el conjunto de datos enriquecido fue importado dentro de Elasticsearch utilizando las herramientas de carga de archivos disponibles en Kibana, permitiendo almacenar e indexar la información generada durante el proyecto por lo que facilitó la integración entre el modelo de Machine Learning y las herramientas de análisis visual, permitiendo utilizar los datos del sistema dentro de dashboards interactivos orientados al análisis del comportamiento estudiantil.

La implementación de Elastic Stack dentro del flujo de trabajo permitió fortalecer los procesos correspondientes a las etapas 6 y 7 del ciclo de vida MLOps, relacionadas con la gestión de datos y modelos mediante de Elasticsearch fue posible centralizar el almacenamiento de la información generada por el modelo, mientras que Kibana permitió construir visualizaciones orientadas al monitoreo y exploración de patrones presentes en los datos con eso el proyecto logró integrar técnicas de Machine Learning con herramientas modernas de análisis y observabilidad, construyendo un entorno más completo para el análisis de interacción estudiantil.

Integración con Elasticsearch y visualización en Kibana

La integración del conjunto de datos con Elasticsearch, se procedió al desarrollo de visualizaciones interactivas mediante Kibana, con el objetivo de representar gráficamente los resultados obtenidos a partir del modelo de Machine Learning y facilitar el análisis del comportamiento estudiantil dentro del entorno digital. El uso de dashboards permitió transformar la información almacenada en Elasticsearch en representaciones visuales más comprensibles e intuitivas, favoreciendo la exploración de patrones de interacción y la interpretación de diferentes métricas relacionadas con la actividad de los estudiantes.

El proceso comenzó con la creación de un data view asociado al índice students_activity, permitiendo que Kibana reconociera las variables almacenadas dentro de Elasticsearch y habilitando su utilización dentro de las herramientas de visualización, después se desarrollaron distintas gráficas

orientadas al análisis de las variables más relevantes del proyecto, incluyendo distribuciones del nivel de actividad, métricas de engagement, tasas de interacción y proporciones de inactividad, permitiendo observar de manera más clara el comportamiento de los estudiantes dentro del sistema. Entre las visualizaciones desarrolladas se incluyó una gráfica de barras orientada a representar la distribución de estudiantes clasificados en niveles de actividad baja, media y alta, permitiendo identificar de manera rápida cuáles categorías presentan mayor frecuencia dentro del conjunto de datos. Asimismo, se construyeron métricas orientadas al análisis del engagement score, facilitando la interpretación del nivel de interacción general de los usuarios y permitiendo comparar diferentes patrones de comportamiento presentes en el dataset. También, se implementaron visualizaciones relacionadas con el interaction rate y el inactivity ratio, variables derivadas que permitieron analizar la intensidad de interacción de los estudiantes y la proporción de tiempo inactivo durante las sesiones. Estas métricas contribuyeron significativamente a enriquecer el análisis visual del proyecto, permitiendo representar relaciones más complejas entre las variables y proporcionando una perspectiva más detallada sobre el comportamiento de los usuarios dentro del entorno digital. Adicionalmente, se desarrollaron visualizaciones tipo heatmap y gráficos de distribución para explorar la relación entre clics, pulsaciones de teclado y niveles de actividad, permitiendo identificar agrupaciones y tendencias presentes en los datos. La posibilidad de aplicar filtros dinámicos sobre las visualizaciones facilitó el análisis interactivo de segmentos específicos del dataset, mejorando considerablemente la capacidad de exploración y monitoreo de la información almacenada dentro de Elasticsearch.

Por último, las visualizaciones fueron integradas dentro de un dashboard general desarrollado en Kibana, permitiendo centralizar las métricas más relevantes del sistema en una sola interfaz interactiva, lo que representa uno de los componentes más importantes del proyecto, debido a que no solo facilita la interpretación de los resultados generados por el modelo de Machine Learning, sino que también fortalece los procesos de monitoreo y gestión de datos dentro del enfoque MLOps, proporcionando una solución más profesional y orientada a la analítica de aprendizaje.

CONCLUSIONES

Resultados del modelo y utilidad práctica

El proyecto permitió demostrar la viabilidad de aplicar técnicas de Machine Learning y analítica de aprendizaje para modelar el comportamiento de estudiantes dentro de entornos digitales de interacción con base de la construcción de un conjunto de datos enriquecido con variables derivadas relacionadas con engagement, interacción e inactividad, fue posible desarrollar un modelo de clasificación capaz de identificar diferentes niveles de actividad estudiantil utilizando métricas provenientes de la interacción del usuario durante sesiones de trabajo computacional. También, la implementación del algoritmo Relief y del modelo de Árbol de Decisión permitió generar un sistema más interpretativo y equilibrado, evitando dependencias excesivas de una sola variable y favoreciendo una clasificación más representativa del comportamiento de los estudiantes.

Valor del análisis visual con Kibana

Las visualizaciones desarrolladas mediante Kibana demostraron ser herramientas altamente efectivas para la exploración y análisis interactivo de los resultados generados por el sistema con el uso de los dashboards dinámicos fue posible representar gráficamente métricas relacionadas con niveles de actividad, engagement score, interaction rate e inactivity ratio, facilitando la interpretación de patrones presentes dentro del conjunto de datos, igualmente la capacidad de aplicar filtros interactivos y explorar segmentos específicos de información permitió mejorar considerablemente el análisis visual del comportamiento estudiantil, fortaleciendo los procesos de monitoreo y gestión de datos dentro del enfoque MLOps.

Aplicaciones futuras y escalabilidad

El enfoque desarrollado durante este proyecto puede ser escalado hacia sistemas de analítica educativa más complejos orientados al monitoreo inteligente del aprendizaje, la detección de patrones de distracción o la identificación temprana de estudiantes con bajo nivel de interacción académica, por lo que el sistema puede evolucionar hacia arquitecturas más avanzadas integrando procesamiento en tiempo real, modelos predictivos más robustos o servicios desplegados mediante APIs y contenedores Docker dentro de entornos distribuidos, de igual forma, las técnicas implementadas podrían adaptarse a otros contextos relacionados con comportamiento humano digital, incluyendo plataformas educativas, sistemas de tutoría inteligente o herramientas de apoyo a procesos de aprendizaje autorregulado.

Reflexión sobre el proceso

El desarrollo del proyecto representó una experiencia integral que combinó preparación y enriquecimiento de datos, selección de características, entrenamiento de modelos de Machine Learning, gestión de proyectos mediante Git y GitHub, implementación de contenedores Docker y construcción de dashboards interactivos utilizando Elastic Stack, gracias a ello permitió comprender de manera práctica diferentes etapas del ciclo de vida MLOps, fortaleciendo tanto la parte técnica como la capacidad de estructurar soluciones orientadas al análisis de datos y visualización profesional de resultados.

ANEXO

En esta sección se incluyen las principales visualizaciones, fragmentos de código y evidencias desarrolladas durante la implementación del proyecto, las cuales complementan el análisis descrito en las secciones anteriores y permiten observar de manera más detallada el funcionamiento del sistema de analítica de aprendizaje basado en Machine Learning y Elastic Stack.

Visualizaciones del dashboard en Kibana

Se presentan las visualizaciones desarrolladas en Kibana a partir de los datos almacenados en Elasticsearch, permitiendo analizar diferentes métricas relacionadas con el comportamiento estudiantil y los niveles de interacción registrados durante las sesiones de trabajo.

La distribución de estudiantes según nivel de actividad.

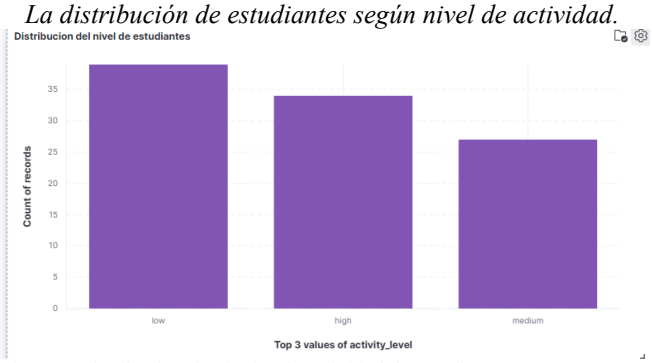

Fig A.1. Distribución de niveles de actividad de estudiantes

El promedio de engagement score por nivel de actividad:

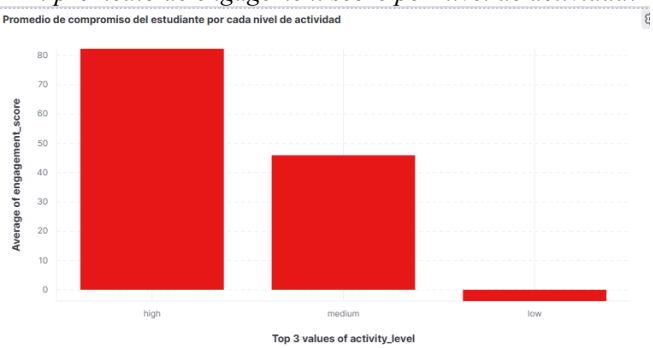

Fig A.2. Promedio de engagement score según nivel de actividad

Distribución de la proporción de inactividad dentro del sistema:

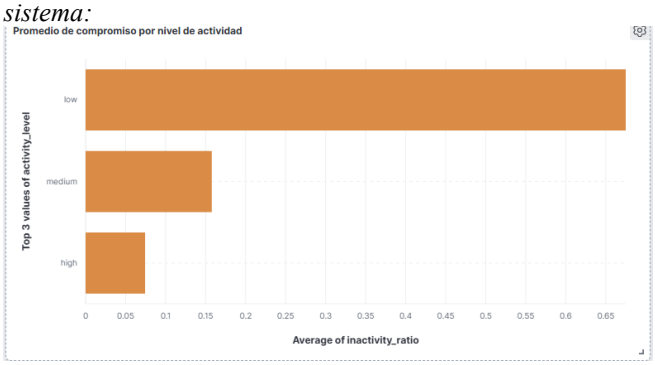

Fig A.3. Distribución del inactivity ratio

Relación entre clics y pulsaciones de teclado mediante heatmap.

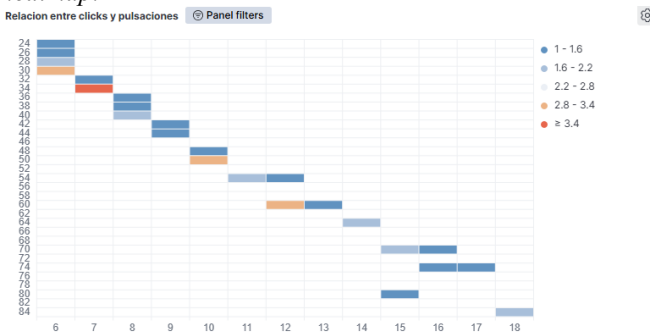

Fig A.4. Relación entre clicks y keystrokes.

Métricas generales del sistema mostradas mediante dashboard interactivo

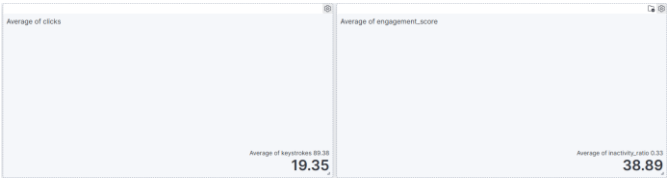

Fig A.5. Dashboard general del sistema de analítica de aprendizaje

Resultados del modelo de Machine Learning

En esta sección se incluyen las principales evidencias relacionadas con el entrenamiento y visualización del modelo de Árbol de Decisión implementado para clasificar el nivel de actividad de los estudiantes.. Se muestran:

Árbol de decisión generado durante el entrenamiento del modelo.

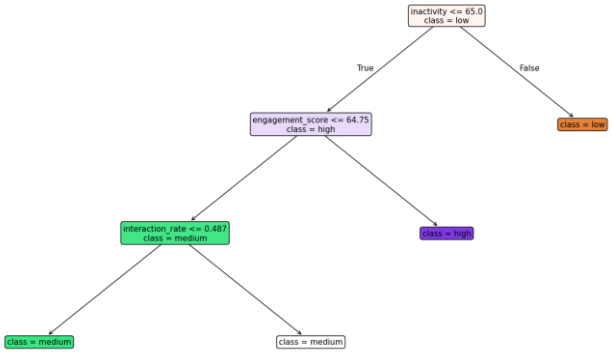

Fig B.1. Árbol de decisión del modelo de clasificación

Importancia de caracteristicas del entrenamiento del modelo..

```
Importancia de características según Relief:
session_time: 0.2497873134328358 6
clicks: 0.2633189655172413
keystrokes: 0.269295
inactivity: 0.3851544117647059
engagement_score: 0.3202632874015748
interaction_rate: 0.09216479238754328
inactivity_ratio: 0.21398285449490265
```

Fig B.2. Caracteristicas en el modelo de clasificación

Reporte de las metricas del modelo.

```
Accuracy del modelo: 0.95

Reporte de clasificación:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         8
           1       1.00      0.80      0.89         5
           2       0.88      1.00      0.93         7

    accuracy                           0.95        20
   macro avg       0.96      0.93      0.94        20
weighted avg       0.96      0.95      0.95        20
```

Fig B.3. Metricas de modelo

Fragmentos de código

Se presentan fragmentos relevantes del código implementado durante las diferentes etapas del proyecto, incluyendo preparación de datos, entrenamiento del modelo y configuración del entorno MLOps.:

Código de preparación y enriquecimiento básico del dataset.:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from skrebate import ReliefF
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt


#df = pd.read_excel("C:/Users/emman/Downloads/database_v1.1.xlsx")
df = pd.read_excel("C:/Users/emman/Downloads/database_v2.0.xlsx")

print("Primeras filas del dataset:")
print(df.head())
```

Fig C.a.1. Código de preparación del conjunto de datos

Implementación del algoritmo Relief para selección de características.

```python
#X = df.drop(["user_id", "activity_level"], axis=1).astype(float)
X = df.drop(["user_id", "activity_level", "activity_numeric" ], axis=1).astype(float)

y = df["activity_level"].map({
    "low": 0,
    "medium": 1,
    "high": 2
})


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTamaño de entrenamiento:", X_train.shape)
print("Tamaño de prueba:", X_test.shape)


relief = ReliefF(n_neighbors=10)
relief.fit(X_train.values, y_train.values)

feature_importance = relief.feature_importances_

print("\nImportancia de características según Relief:")
for feature, importance in zip(X.columns, feature_importance):
    print(f"{feature}: {importance}")
```

Fig C.b.1. Código de selección de características mediante Relief

Entrenamiento del modelo de Árbol de Decisión.

```python
model = DecisionTreeClassifier(
    random_state=42,
    max_depth=5,
    min_samples_split=4,
    min_samples_leaf=2
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)


accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy del modelo:", accuracy)

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred))


plt.figure(figsize=(20, 10))

annotations = plot_tree(
    model,
    feature_names=X.columns,
    class_names=["low", "medium", "high"],
    filled=True,
    rounded=True,
    impurity=False,
    fontsize=11
)

for ann in annotations:
    text = ann.get_text().split("\n")

    if len(text) > 1 and "<=" in text[0]:
        new_text = text[0] + "\n" + text[-1]
    else:
        new_text = text[-1]

    ann.set_text(new_text)

plt.title("Árbol de Decisión", fontsize=16)
plt.tight_layout()
plt.savefig("decision_tree_clean.png", dpi=300, bbox_inches="tight")
plt.show()
```

Fig C.c.1. Código de entrenamiento del modelo de clasificación

Configuración de Docker Compose para Elastic Stack.

```
hadoop@Ubuntu: ~
  GNU nano 6.2                    docker-compose.yml
version: '3.7'

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.5.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
    ports:
      - "9200:9200"

  kibana:
    image: docker.elastic.co/kibana/kibana:8.5.0
    container_name: kibana
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch

                          [ Read 19 lines ]
^G Help     ^O Write Out ^W Where Is  ^K Cut     ^T Execute   ^C Location
^X Exit     ^R Read File ^\ Replace   ^U Paste   ^J Justify   ^/ Go To Line
```

Fig C.2. Configuración de Docker Compose para Elastic Stack

Carga del dataset enriquecido dentro de Elasticsearch mediante Kibana.

Type	Name	Documents (%)	Distinct values	Distributions		
	activity_level	100 (100%)	3	3 categories		
	activity_numeric	100 (100%)	3	min 1	median 2	max 3
	clicks	100 (100%)	29	min 6	median 20	max 35
	engagement_score	100 (100%)	72	min -27.5	median 42.5	max 99.5
	inactivity	100 (100%)	24	min 15	median 45	max 100
	inactivity_ratio	100 (100%)	76	min 0.04	median 0.15	max 1.12
	interaction_rate	100 (100%)	81	min 0.17	median 0.46	max 1.04
	keystrokes	100 (100%)	35	min 25	median 95	max 150
	session_time	100 (100%)	75	min 85	median 267.5	max 420
	user_id	100 (100%)	100	min 1	median 50.5	max 100

Rows per page: 25 < 1 >

Fig C.3. Importación del dataset en Kibana

Infraestructura y entorno de trabajo

Se incluyen evidencias relacionadas con la configuración del entorno de desarrollo utilizado durante el proyecto.

Verificación de instalación y funcionamiento de Docker en Linux.

```
hadoop@Ubuntu:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Fig D. 1. Verificación de Docker en máquina virtual Linux

Contenedores de Elasticsearch y Kibana ejecutándose correctamente.

```
hadoop@Ubuntu:~$ docker-compose up -d
Starting elasticsearch ... done
Starting kibana        ... done
hadoop@Ubuntu:~$ docker ps
CONTAINER ID   IMAGE                                                   COMMAND
                 CREATED      STATUS          PORTS
                      NAMES
06300d8a63e1   docker.elastic.co/kibana/kibana:8.5.0                   "/bin/tini
-- /usr/l…"   3 days ago   Up 54 seconds   0.0.0.0:5601->5601/tcp, [::]:5601->56
01/tcp             kibana
a84d15fc6982   docker.elastic.co/elasticsearch/elasticsearch:8.5.0   "/bin/tini
-- /usr/l…"   3 days ago   Up 54 seconds   0.0.0.0:9200->9200/tcp, [::]:9200->92
00/tcp, 9300/tcp _ elasticsearch
```

Fig D. 1. Contenedores Docker activos

Interfaz principal de Kibana utilizada para la construcción del dashboard.

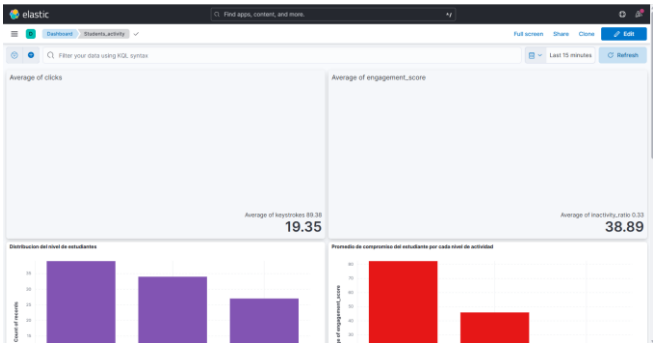

Fig D. 1. Interfaz de Kibana y visualización de datos

REFERENCIAS

Link de GitHub:
https://github.com/Caleb234Perez/tutor_metacognitivo.git

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove template text from your paper may result in your paper not being published